Non-Linear Data Structure: Trees

CSE 4303: Data Structures
Lecture 5

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

November 24, 2025

Recap: Linear Data Structures

- **What we know:** Arrays, Linked Lists, Stacks, and Queues.
- **Characteristics:**
  - Data elements are arranged sequentially one after another.
  - Single level of storage.
  - Each element is connected to exactly one previous and one next element (except ends).
- **Relationship:** 1:1 Relationship.

Linear structures are excellent for sequential data, but is all data sequential?

The Limitation: When Linear Fits Poorly

Scenario 1: Organizational Hierarchy

- **Reality:** A CEO manages 2 VPs. Each VP manages 3 Managers.
- **Linear Attempt:** `[CEO, VP1, VP2, Mgr1, Mgr2, ...]`

The Problem

- **Loss of Relationship:** Who reports to whom? Is Mgr1 under VP1 or VP2?
- **Insertion Difficulty:** If VP1 hires a new Manager, we must shift the entire array to keep them physically close, or lose the logical grouping entirely.

Scenario 2: Where Linear Fails

Linear structures (Lines) cannot naturally represent:

1. File Systems

- 👉 Root Drive (C:) → Users → Documents.
- 👉 Files are *nested*, not sequential.

2. Web Pages (DOM)

- 👉 `<HTML>` contains `<BODY>`, which contains `<DIV>`, etc.

3. Game AI (Decisions)

- 👉 If *Health* $< 10\%$ → *Run Away*.
- 👉 Else → *Attack*.
- 👉 Represents a branching path of choices.

Conclusion: We need a structure that supports *branching*.

Enter: Non-Linear Data Structures

- **Definition:** Data structure where data elements are not arranged sequentially or linearly.
- **Characteristics:**
  - Single element can be connected to multiple elements.
  - Represents hierarchical (1:N) or interconnected (N:N) relationships.
  - Examples: Trees and Graphs.

Linear (Line) vs. Non-Linear (Branching)

Why Non-Linear?

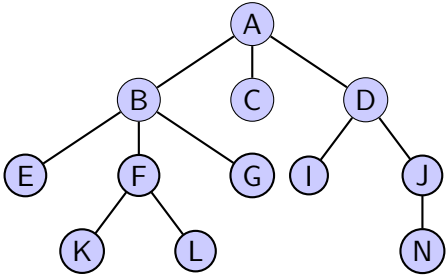
1. **Reflecting Reality:** Many real-world data sets are hierarchical (e.g., Folder structures, HTML DOM, Decision making).
2. **Efficiency:**
   - **Search:** Linear search is $O(N)$. Specialized trees (like BST) can search in $O(\log N)$.
   - **Access:** Faster access to related elements without traversing unrelated ones.
3. **Organization:** Handles complex relationships that cannot be represented by simple sequence.

What is a Tree?

Definition

A **Tree** is a non-linear data structure defined as a finite set of one or more nodes such that:

1. There is a distinct node called the **Root**.
2. The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, T_2, \ldots, T_n$, where each of these sets is itself a tree.



- These sets $T_1, \ldots, T_n$ are called the **Subtrees** of the root.
- This is a *recursive* definition.

What does a Tree Represent?

Trees represent **hierarchical relationships** (Parent-Child relationships).

Key Characteristics:

- 👉 **Top-down structure:** Data flows from the Root downwards.
- 👉 **No Cycles:** There are no loops; you cannot get back to the starting node by following connections downwards.
- 👉 **Connectivity:** Every node (except the root) has exactly one parent.

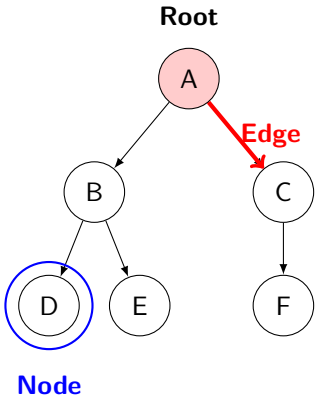
Think of it as an inverted natural tree: Root at the top, leaves at the bottom.

Why do we need Trees?

1. **Hierarchical Storage:** To store data that naturally forms a hierarchy (e.g., File systems, HTML, Taxonomy).
2. **Efficient Search:**
   - *Binary Search Trees (BST)* allow searching, insertion, and deletion in $O(\log n)$ time (average).
   - Much faster than Linked Lists $O(n)$ and Arrays (for insertion/deletion).
3. **Sorting and Indexing:** Used in Databases (B-Trees) and Compilers (Syntax Trees).
4. **Networking:** Routing algorithms and Spanning trees.

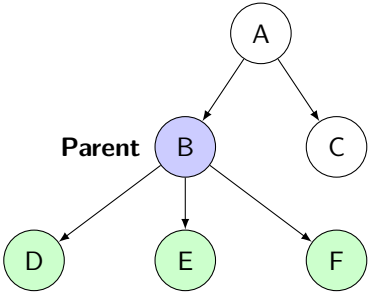
Terminologies: Basic Components

- **Node:** The fundamental building block storing data.
- **Root:** The unique, topmost node in the tree. It has no parent.
- **Edge (Branch):** The connecting link between two nodes, representing the relationship.



Terminologies: Relationships

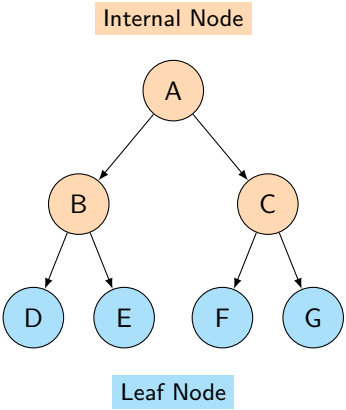
- **Parent:** A node that has a successor (child).
- **Child:** A node that is the successor of another node (parent).
- **Siblings:** Nodes that share the <u>same</u> parent.



Children of B & Siblings

Terminologies: Node Types

- **Leaf Node (External Node):** A node with **no** children. It is at the "bottom" of the tree.
- **Internal Node:** A node with **at least one** child. The Root is usually an internal node (unless it's the only node).

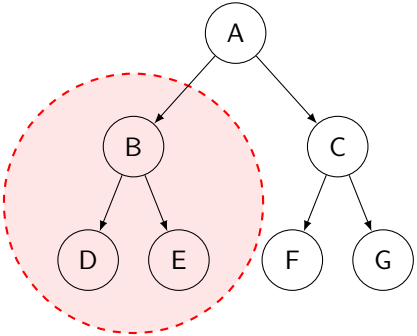


Terminologies: Subtree

Definition

A **Subtree** is any node in the tree along with all of its descendants.

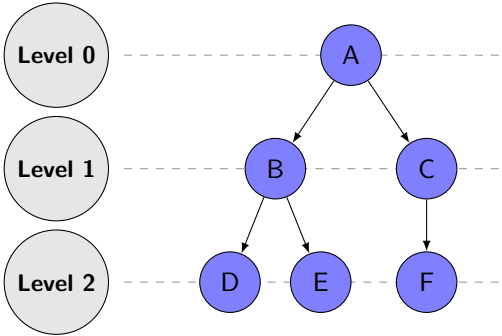
👉 Every child of a node forms the root of a new subtree.



Subtree rooted at B

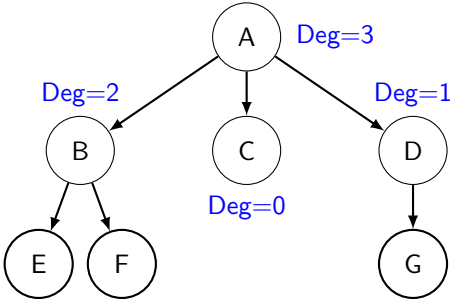
Terminologies: Level Number

- The **Level** represents the generation or distance from the root.
- **Convention:** The Root is at **Level 0**.
- Children of Level i are at Level $i + 1$.



Terminologies: Degree Concepts

- **Degree of a Node:** The number of children (subtrees) a node has.
- **Degree of a Tree:** The maximum degree found among any node in the tree.
- **In-degree:** Incoming edges (Always 1, except Root=0).
- **Out-degree:** Outgoing edges (Same as Degree).



Degree of Tree = 3 (Node A)

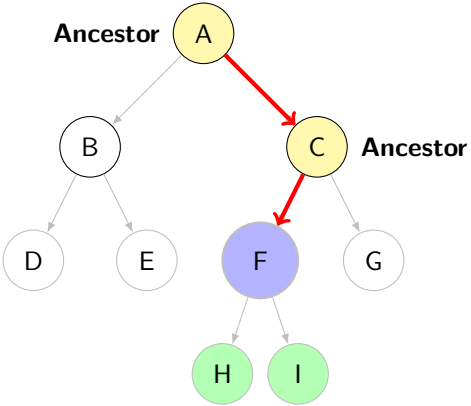
Important Note

In **Graph Theory**, degree is the total connections. In **Trees**, "Degree" specifically refers to the number of branches going *downwards* (Out-degree).

Terminologies: Lineage Path

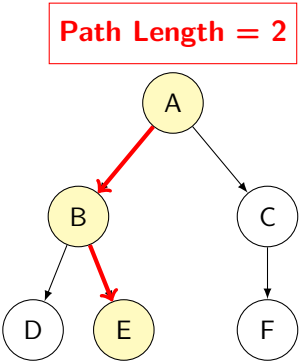
- **Ancestors:** All nodes along the path from the Root to th specific node.
- **Descendants:** All nodes in the subtree rooted at the specific node.

*Ancestors of **F**: A, C*
*Descendants of **F**: H, I*



Terminologies: Path

- **Path:** A sequence of consecutive edges from a source node to a destination node.
- **Path Length:** The number of *edges* on the path.
  - Note: Some texts count nodes, but standard convention (and C implementations) usually counts edges.
- **Example:** Path from A to E.
- Sequence: $A \to B \to E$
- Length: 2 Edges.

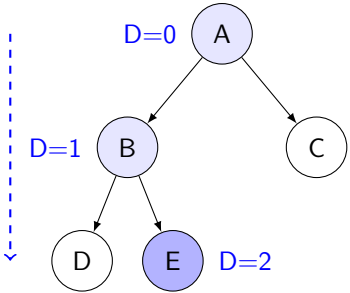


Terminologies: Depth

Definition

The **Depth** of a node is the number of edges from the **Root** to that node.

- **Direction:** Calculated Top-Down ↓.
- **Root Depth:** Always 0.
- **Formula:** Depth(Node) = Depth(Parent) + 1.

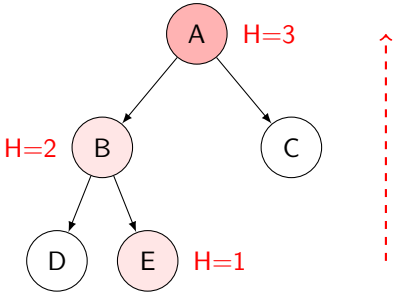


Terminologies: Height

Definition

The **Height** of a node is the number of nodes on the *longest* path from that node to a **Leaf**.

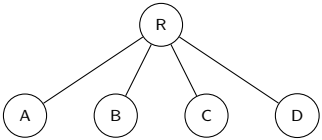
- **Direction:** Calculated Bottom-Up ↑.
- **Leaf Height:** Always 1.
- **Height of Tree:** The Height of the Root node.



Types: General Tree & Forest

1. General Tree

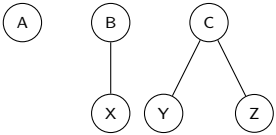
- A tree where there is **no limit** on the number of children a node can have.
- Degree can be any non-negative integer.



General Tree (Degree 4)

2. Forest

- A collection of disjoint trees.
- If you delete the Root of a tree, the subtrees form a Forest.
- $F = \{T_1, T_2, \ldots, T_n\}$
- By set theory it can also be empty.



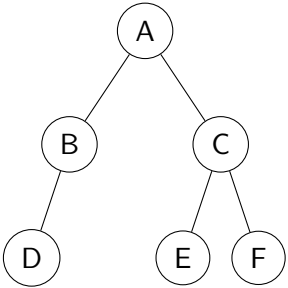
Forest (3 Disjoint Trees)

Types: Binary Tree

Definition

A **Binary Tree** is a tree where every node has **at most two** children.

- Children are specifically named:
  1. **Left Child(Successor)**
  2. **Right Child(Successor)**
- Degree ≤ 2.
- Basis for many efficient algorithms.



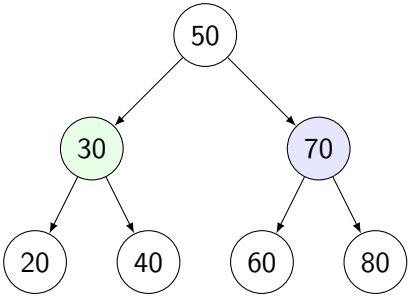
Types: Binary Search Tree (BST)

Purpose: Optimized for Searching ($O(\log n)$).

The BST Property

For any node N:

- 👉 Values in the **Left Subtree** are **smaller** than N.
- 👉 Values in the **Right Subtree** are **larger** than N.



Left Subtree < 50 Right Subtree > 50

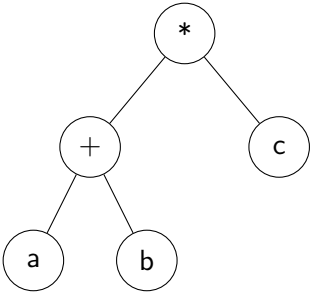
Types: Expression Tree

Purpose: Used by compilers to parse expressions.

Expression Tree Property

- **Internal Nodes:** Operators $(+, -, *, /)$.
- **Leaf Nodes:** Operands (Variables or Constants).
- **Structure:** Represents the order of operations.

Example Expression: $(a + b) * c$

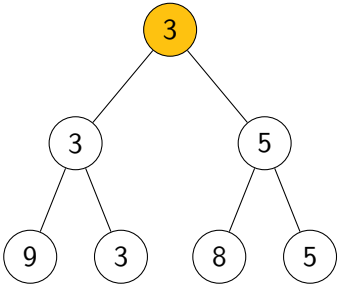


Types: Tournament Tree (Winner Tree)

Purpose: Used in selection algorithms and sorting (Heapsort variation).

Tournament Tree Property

- Represents a competition structure.
- **Leaf Nodes:** Players (or initial values).
- **Internal Nodes:** The winner of the match between children.



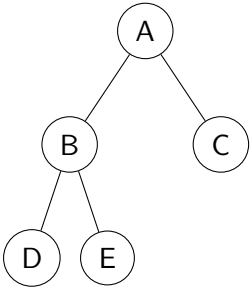
Min-Tournament: Smaller value wins

Variations: Strictly Binary Tree

Definition

A **Strictly Binary Tree** (also called a **Full Binary Tree** or **2-Tree**) is a binary tree where every node has either **0** or **2** children.

- No node has degree 1.
- Used in Huffman Coding and Expression Trees.



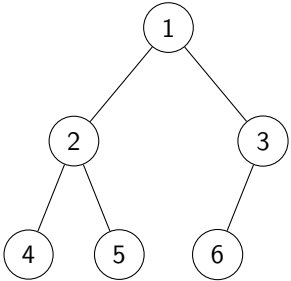
Valid: Nodes have 0 or 2 children.
Invalid: A node having only 1 child.

Variations: Complete Binary Tree

Definition

A **Complete Binary Tree** satisfies two conditions:

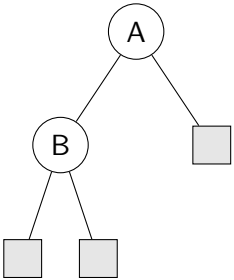
1. All levels are completely filled, except possibly the last level.
2. In the last level, all nodes are as far **left** as possible.



Essential for Heap Data Structures (Array representation).

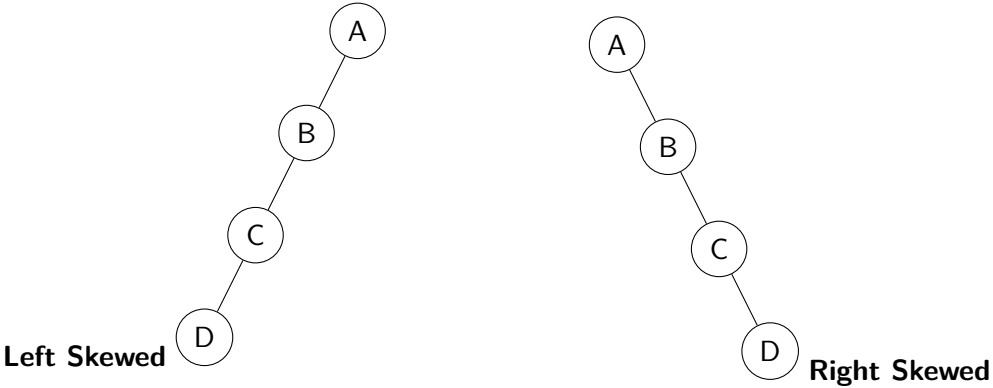
Variations: Extended Binary Tree

- **Concept:** Converting any binary tree into a Strictly Binary Tree by adding special nodes.
- **Internal Nodes (Circles):** The original nodes.
- **External Nodes (Squares):** Dummy nodes added wherever a null subtree existed.
- (External path length is related to Internal path length).



Variations: Skewed Binary Trees

Skewed Trees arise when every node has only one child.

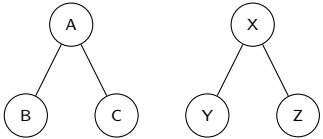


Performance Hit: Search becomes $O(n)$ instead of $O(\log n)$. Effectively a Linked List.

Relationships Between Trees

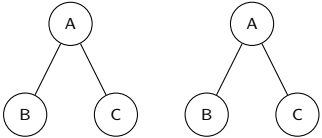
1. Similar Binary Trees

- 👉 Have the exact same **structure** (shape).
- 👉 Values in the nodes may differ.



2. Equivalent Trees (Copies)

- 👉 Have the same **structure**.
- 👉 Have the same **data** in corresponding nodes.
- 👉 They are identical clones.



Properties: Extended Binary Tree ($E = I + 1$)

Theorem

In any Extended Binary Tree (Strictly Binary), the number of **External Nodes (E)** is always one greater than the number of **Internal Nodes (I)**.

Intuitive Proof (Induction):

1. **Base Case:** A single root node.
   - $I = 0$, $E = 1$ (The root is external).
   - $1 = 0 + 1$ ✓
2. **Step:** Take an external node and make it internal (give it 2 children).
   - I increases by 1 (the node becomes internal).
   - E decreases by 1 (node lost) but increases by 2 (new children).
   - Net change in $E = -1 + 2 = +1$.

Since both I and E increase by 1 at each step, the difference remains constant.

Properties: Height vs Node Count

Let H be the **Height** of the tree (counting nodes, Root at $H = 1$).

Inverse Relationship (Nodes from Height):

- 👉 **Min Nodes:** $n_{min} = H$ (Linear/Skewed Tree).
- 👉 **Max Nodes:** $n_{max} = 2^H - 1$ (All levels are full).

Inverse Relationship (Height from Nodes):

- 👉 **Min Height:** $H_{min} = \lfloor \log_2 n \rfloor$ (Best case: Complete Tree).
- 👉 **Max Height:** $H_{max} = n$ (Worst case: Skewed Tree).

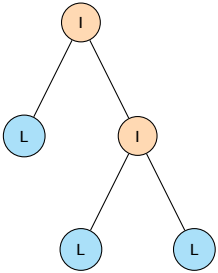
Significance

This explains why BST search is $O(\log n)$ in the best case but $O(n)$ in the worst case.

Properties: Strictly Binary Tree

Recall: Every node has 0 or 2 children.

- **Total Nodes (n) is always Odd.**
  - $n = 2 \times I + 1$
- **Leaf Nodes (L) vs Internal Nodes (I):**
  - $L = I + 1$ (Same as $E = I + 1$).
- **Leaf Nodes (L) vs Total Nodes (n):**
  - $L = (n + 1)/2$



$I = 2, L = 3 \rightarrow L = I + 1$

Proof: $n = 2I + 1$

Theorem: In a Strictly Binary Tree, Total Nodes $(n) = 2\times$ Internal Nodes $(I) + 1$.

Proof using Edge Counting

1. **Step 1: Count Edges by Destination**
   - In any tree, every node (except the Root) has exactly 1 incoming edge.
   - **Total Edges** $E_{total} = n - 1$.
2. **Step 2: Count Edges by Source**
   - Edges originate only from Internal Nodes.
   - In a Strictly Binary Tree, every Internal Node creates exactly **2** edges.
   - **Total Edges** $E_{total} = 2 \times I$.
3. **Step 3: Equate them**
   - $n - 1 = 2I \qquad => \qquad n = 2I + 1$

Q.E.D. (Total nodes is always an odd number)

Properties: Complete Binary Tree

A Complete Binary Tree is packed as densely as possible.

👉 **Height is logarithmic:**

$$H = \lfloor \log_2 n \rfloor$$

👉 **Array Indexing (1-based index):**

- If Node is at index i:
- **Left Child:** $2 \times i$
- **Right Child:** $(2 \times i) + 1$
- **Parent:** $\lfloor i/2 \rfloor$

Why this matters?

This property allows Complete Binary Trees to be stored efficiently in **Arrays** without gaps, bypassing the need for pointers (e.g., Heaps).

Representation of Binary Trees

Computer memory is linear. To store a non-linear tree, we have two main approaches:

1. Sequential Representation

- 👉 Uses **Arrays**.
- 👉 Nodes are stored at specific indices based on their position in a complete tree.
- 👉 Best for: Complete Binary Trees (Heaps).

2. Linked Representation

- 👉 Uses **Pointers** and Dynamic Memory.
- 👉 Each node contains data and links to children.
- 👉 Best for: General/Skewed Trees.

1. Sequential (Array) Representation

Logic: We map the tree nodes onto an array index i using a formula.

Indexing Formulas (0-based Index for C)

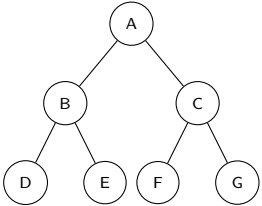
If a node is at index i:

- 👉 **Left Child:** $2i + 1$
- 👉 **Right Child:** $2i + 2$
- 👉 **Parent:** $\lfloor \frac{i-1}{2} \rfloor$ (if $i > 0$)

Note: If using 1-based indexing (Root=1), Left is $2i$, Right is $2i + 1$.

Sequential Representation: Visual Example

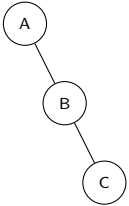
Complete Tree (Efficient)



`[A, B, C, D, E, F, G]`

No wasted space

Skewed Tree (Inefficient)



`[A, _, B, _, _, _, C]`

Wasted memory gaps

Disadvantage

For skewed trees, array size grows exponentially (2^h), leaving many empty slots.

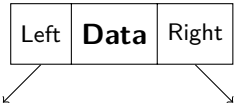
2. Linked Representation

Structure: We use a self-referential structure (like a Doubly Linked List).

C Structure Definition:

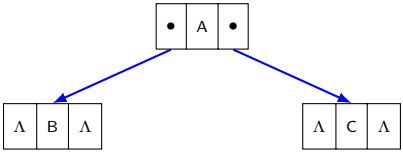
```
struct Node {
    struct Node *left;
    int data;
    struct Node *right;
};
```

Visual Node:



Linked Representation in Memory

- Memory is allocated dynamically (`malloc`).
- Nodes are scattered in memory, connected only by addresses.



Λ (NULL) indicates no child.

Comparison: Array vs Linked

Feature	Sequential (Array)	Linked (Pointer)
Access	Direct Access (Index)	Sequential Access (Traversal)
Memory	Fixed size (Static)	Dynamic size
Efficiency	Wasteful for sparse trees	Efficient for all types
Complexity	Easy impl. for Heaps	Higher overhead (pointers)

Binary Tree Traversal

Definition

Traversal is the process of visiting every node in a tree data structure exactly once in a systematic way.

Why is it different from Arrays?

- 👉 Arrays have a linear order (index $0 \to n$).
- 👉 Trees are non-linear; there is no unique "next" node.
- 👉 We must define an order.

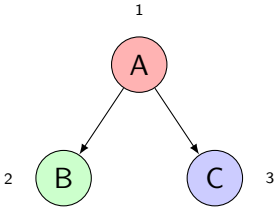
The Three Standard Approaches (DFS):

1. **Pre-order** (Root $\to$ Left $\to$ Right)
2. **In-order** (Left $\to$ Root $\to$ Right)
3. **Post-order** (Left $\to$ Right $\to$ Root)

1. Pre-order Traversal (N-L-R)

Algorithm:

1. Visit the **Root** (N).
2. Traverse the **Left** subtree recursively (L).
3. Traverse the **Right** subtree recursively (R).



👉 **Use Case:**

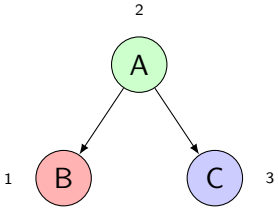
- Creating a copy of the tree
- Prefix expression (Polish notation) from expresison tree

👉 **Trace(Order of Traversal):** A → B → C

2. In-order Traversal (L-N-R)

Algorithm:

1. Traverse the **Left** subtree recursively (L).
2. Visit the **Root** (N).
3. Traverse the **Right** subtree recursively (R).



👉 **Use Case:**

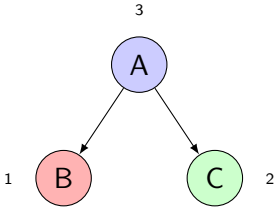
- Returns values in non-decreasing order for Binary Search Trees (BST)
- Infix expression generation from expression tree

👉 **Trace (order of traversal):** B → A → C

3. Post-order Traversal (L-R-N)

Algorithm:

1. Traverse the **Left** subtree recursively (L).
2. Traverse the **Right** subtree recursively (R).
3. Visit the **Root** (N).



👉 **Use Case:**

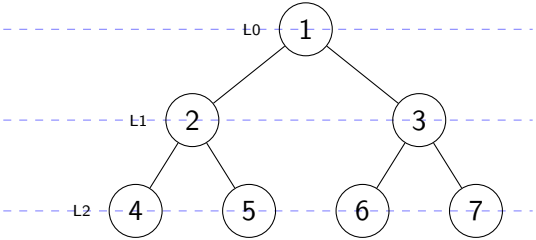
- Deleting the tree (must delete children before deleting parent)
- Postfix expression (Reverse Polish Notation) from expression tree

👉 **Trace (order of traversal):** B → C → A

4. Level Order Traversal (BFS)

Definition

Level Order Traversal visits nodes level-by-level, starting from the Root (Level 0), then Level 1, Level 2, and so on.



Characteristics:

- **Direction:** Top-to-Bottom, Left-to-Right within each level.
- **Data Structure Used: Queue** (FIFO).
- **Recursion?** No. Standard implementation is iterative.

Order:
Level 0: 1
Level 1: 2, 3
Level 2: 4, 5, 6, 7
Result: 1, 2, 3, 4, 5, 6, 7

Level Order: How it works (Queue)

Algorithm Steps:

1. Enqueue Root.
2. While Queue is not empty:
   - Dequeue a node N and visit it (print it).
   - Enqueue N's Left Child (if exists).
   - Enqueue N's Right Child (if exists).

Trace Example:

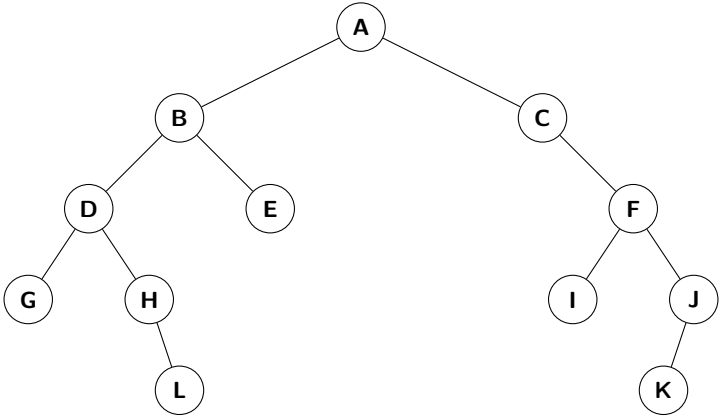
Q1: [1]

Q2: [2, 3] (Removed 1, Added Children)

Q3: [3, 4, 5] (Removed 2, Added Children)

Q4: [4, 5, 6, 7] (Removed 3, Added Children)

Class Exercise



Reconstructing a Tree

Problem: Given a sequence of node values from a traversal, can we reconstruct the unique Binary Tree?

The Rule

A single traversal is **not enough** to construct a unique tree (unless it is a BST).

- 👉 You need **two** traversals.
- 👉 One of them **MUST be In-order**.

Valid Combinations:

1. **Pre-order** + **In-order** ✓
2. **Post-order** + **In-order** ✓
3. Pre-order + Post-order × (Ambiguous)

Logic: Pre-order + In-order

Given:

- 👉 **Pre-order:** [Root, Left, Right] → First element is always Root.
- 👉 **In-order:** [Left, Root, Right] → Root divides Left and Right subtrees.

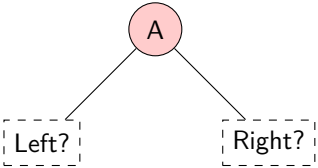
Algorithm Steps:

1. Pick the **first** element from Pre-order. This is the **Root**.
2. Find this Root value in the In-order sequence.
3. Everything to the **left** of Root in In-order becomes the **Left Subtree**.
4. Everything to the **right** of Root in In-order becomes the **Right Subtree**.
5. Repeat recursively for subtrees.

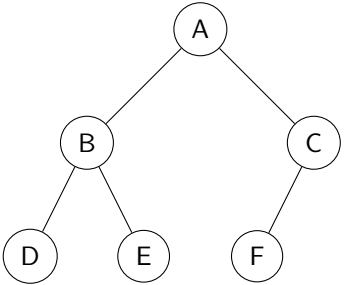
Worked Example: Pre + In

Input:

- 👉 Pre-order: `[A, B, D, E, C, F]`
- 👉 In-order: `[D, B, E, A, F, C]`



1. Pre: **A** is first → Root.
2. In: [D, B, E] **A** [F, C]
3. Left: {D, B, E}, Right: {F, C}



4. Next in Pre is **B** (Left root).
5. In: [D] **B** [E]
6. Next in Pre is **C** (Right root).
7. In: [F] **C** []

Logic: Post-order + In-order

Given:

- **Post-order:** [Left, Right, Root] → Last element is Root.
- **In-order:** [Left, Root, Right]

Key Difference:

- Read Post-order from **Right to Left** (End to Start).
- The last item is the Root. The second to last is the root of the Right subtree.

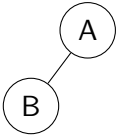
Example: Post=[D, E, B, F, C, A], In=[D, B, E, A, F, C]
Root is **A**. Then look at **C** (Right child), then **B** (Left child).

Why is one traversal insufficient?

The Problem: A single traversal sequence (like Pre-order) does not preserve unique structural information.

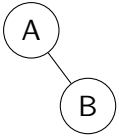
Counter-Example: Consider Pre-order sequence `[A, B]`. This could represent two completely different trees:

Tree 1 (Left Skewed)



Pre-order: **A, B**
In-order: **B, A**

Tree 2 (Right Skewed)



Pre-order: **A, B**
In-order: **A, B**

Result: Without a second sequence, we cannot know if 'B' is a Left child or a Right child.

Why do we strictly need In-order?

- **Pre-order / Post-order / Level-order** tell us **WHO** is the Root.
  - Pre: First is Root.
  - Post: Last is Root.
- **In-order** tells us **WHERE** nodes are relative to the Root.
  - It acts as a **Separator**.

This boundary is only provided by In-order

Left Subtree	**ROOT**	Right Subtree

Structure of In-order Array

Exception: If we know the tree is a Binary Search Tree (BST), one traversal (Pre or Post) is sufficient, because the "separator" logic is implied by values (< `Root` is Left, > `Root` is Right).

Huffman Tree

Definition

A **Huffman Tree** is a weighted, strictly binary tree used to determine the optimal prefix-free codes for data compression.

Key Characteristics:

- 👉 **Weighted:** Leaf nodes carry "weights" (frequency of characters).
- 👉 **Strictly Binary:** Every node has 0 or 2 children.
- 👉 **Optimality:** More frequent characters get shorter codes (closer to the root). Less frequent characters get longer codes.

Huffman Construction Algorithm

Strategy: Greedy Algorithm (Bottom-Up).

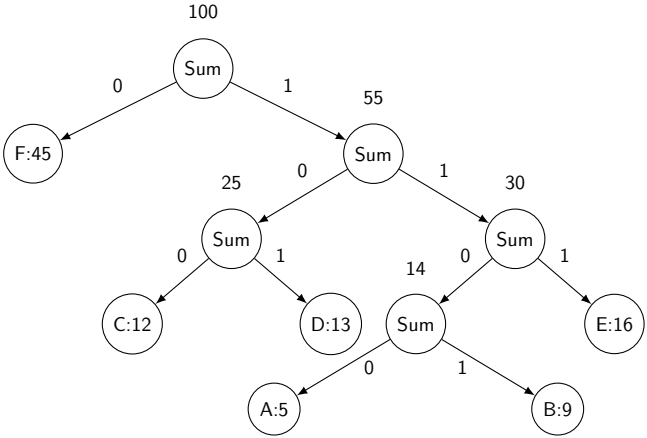
Steps:

1. Create a leaf node for each character and queue them by frequency (weight).
2. While there is more than one node in the queue:
   - Remove the two nodes with the **lowest** frequencies (min_1, min_2).
   - Create a new internal parent node with weight $= weight(min_1) + weight(min_2)$.
   - Make min_1 and min_2 children of this new node.
   - Insert the new node back into the queue.
3. The remaining node is the Root.

Huffman Tree Example

Input:
A: 5, B: 9, C: 12, D: 13, E: 16, F: 45

Code Generation: Trace root to leaf.
F=0, C=100, D=101, A=1100 ...



The Prefix Property

Why trees?

- 👉 In Huffman coding, no code is a **prefix** of another.
- 👉 Example: If A=0, then no other character can start with 0 (like 01 or 001).

Tree Visualization

This is naturally enforced by the tree structure.

- 👉 Characters are only at the **Leaves**.
- 👉 You cannot pass through a leaf to get to another node.
- 👉 Therefore, no code "contains" another.

Huffman Efficiency: The Calculation

Scenario: We transmit a file with 100 characters using the frequencies from the previous slide.

1. **Uncompressed (Fixed-Length)**
   - 👉 Standard ASCII (8 bits/char).
   - 👉 Total Size: 100 chars $\times$ 8 bits $=$ $\mathbf{800}$ bits.
2. **Huffman (Variable-Length)**

Char	Freq	Code Len	Total Bits
F	45	1	45
E	16	3	48
D	13	3	39
C	12	3	36
B	9	4	36
A	5	4	20
Sum	**100**	-	**224 bits**

Compression Ratio

$$\text{Saved} = \left(1 - \frac{\text{Compressed}}{\text{Original}}\right) \times 100\% = 72\%$$

Note: Even if we used a minimal 3-bit fixed code ($\lceil \log_2 6 \rceil = 3$):

- 👉 Fixed Size: $100 \times 3 = 300$ bits.
- 👉 Savings: $1 - (224/300) \approx \mathbf{25}\%$.

Huffman Decompression (Decoding)

The Key Requirement: To decode the bit stream, the receiver must have the **same Huffman Tree** used for compression.

Decoding Algorithm

1. Start at the **Root** of the Huffman Tree.
2. Read the input bits one by one:
   - If Bit == **0**: Move to the **Left Child**.
   - If Bit == **1**: Move to the **Right Child**.
3. **Check:** If the current node is a **Leaf Node**:
   - Output the character stored in the leaf.
   - Reset the pointer back to the **Root**.
4. Repeat until all bits are processed.

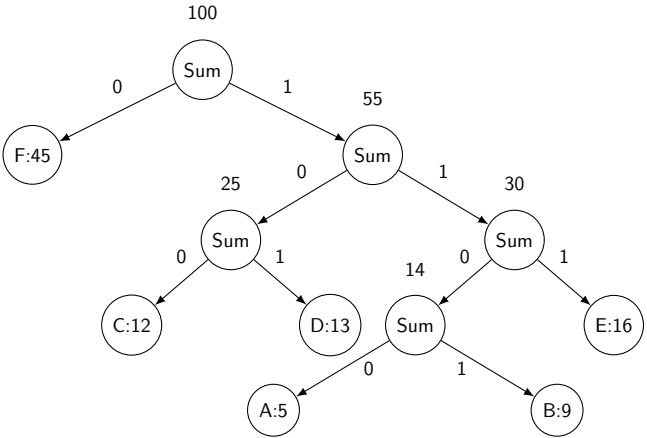
Decompression: Worked Example

Input Stream: 1 1 0 0 0

Trace Steps:

1. **Bit 1:** Right → (Node R1)
2. **Bit 1:** Right → (Node R2)
3. **Bit 0:** Left → (Node R3)
4. **Bit 0:** Left → **Leaf A!**
   - Output: 'A'
   - Reset to Root.
5. **Bit 0:** Left → **Leaf F!**
   - Output: 'F'
   - Reset to Root.

Result: "AF"



🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 9**:(Covered topics in the slides only)

Thank You!

👏 Thank you for your time and attention. 👏

🗨 Any questions or clarifications. 💭